



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Dr. Mohammed Razia Alangir Banu
Student Name	Rayanki Himavarshini
Roll Number	2520030141
Branch	CSE
Course Outcome	CO6

Case Study Topic: AgroChain – Smart Agriculture Supply Chain Management System

Work Done Summary:

In the **AgroChain – Smart Agriculture Supply Chain Management System**, Greedy Algorithms and Dynamic Programming techniques (Memoization and Tabulation) were implemented to solve real-world optimization problems related to agricultural production, inventory management, transportation, and resource allocation.

The system utilized **Activity Scheduling, Shortest Path, 0/1 Knapsack, Longest Common Subsequence (LCS), and Matrix Chain Multiplication (MCM)** to improve supply chain efficiency and decision-making.

Implementation Details:

Applied **Activity Scheduling (Greedy Strategy)** to optimize farming operations, harvesting schedules, and transportation activities.

- Implemented **Shortest Path Algorithm** to determine the most efficient transportation routes from farms to markets.
- Used **0/1 Knapsack Problem (Dynamic Programming)** to maximize transportation profit under vehicle capacity constraints.
- Applied **Longest Common Subsequence (LCS)** to compare agricultural records, crop histories, and supply chain documents.
- Implemented **Matrix Chain Multiplication (MCM)** to optimize large-scale agricultural data processing operations.
- Utilized **Memoization** to store intermediate results and reduce redundant computations.

- Utilized **Tabulation** to solve optimization problems iteratively with improved performance.
- Analyzed overlapping subproblems and optimal substructure properties for selecting appropriate algorithms.
- Evaluated execution efficiency and solution quality on agricultural datasets.
- Generated optimized schedules, routes, and resource allocation plans.

Input Screenshot :

```
import java.util.*;

class Activity {
    int start, finish;

    Activity(int start, int finish) {
        this.start = start;
        this.finish = finish;
    }
}

public class GreedyActivitySelector {
    // Activity Scheduling (Greedy)
    static void activitySelector(Activity activities[]) {
        Arrays.sort(activities, Comparator.comparingInt(a -> a.finish));

        System.out.println("Selected following activities:");

        int lastFinish = activities[0].finish;
        System.out.println("(" + activities[0].start + ", " +
            activities[0].finish + ")");

        for (int i = 1; i < activities.length; i++) {
            if (activities[i].start >= lastFinish) {
                System.out.println("(" + activities[i].start + ", " +
                    activities[i].finish + ")");
                lastFinish = activities[i].finish;
            }
        }
    }
}
```

```
// Q1 Knapsack (Dynamic Programming)
static int knapsack(int capacity, int weights[],
    int profits[], int n) {

    int dp[][] = new int[n + 1][capacity + 1];

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {

            if (i == 0 || w == 0)
                dp[i][w] = 0;

            else if (weights[i - 1] <= w)
                dp[i][w] = Math.max(
                    profits[i - 1]
                    + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]);

            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    return dp[n][capacity];
}
```

```

public static void main(String[] args) {

    // Activity Scheduling Example
    Activity activities[] = {
        new Activity(1, 3),
        new Activity(2, 5),
        new Activity(4, 7),
        new Activity(1, 8),
        new Activity(8, 9)
    };

    activitySelection(activities);

    // Knapsack Example
    int profits[] = {60, 100, 120};
    int weights[] = {10, 20, 30};
    int capacity = 50;

    int maxProfit = knapsack(
        capacity, weights, profits, profits.length);

    System.out.println("\nMaximum Transport Profit: "
        + maxProfit);
}
}

```

Output Screenshot :

```

Selected Farming Activities:
(1, 3)
(4, 7)
(8, 9)

Maximum Transport Profit: 220

```

Result :

The AgroChain system successfully optimized agricultural supply chain operations using Greedy and Dynamic Programming techniques. Activity Scheduling improved resource utilization, Shortest Path algorithms reduced transportation costs, 0/1 Knapsack maximized cargo value, LCS enhanced record comparison accuracy, and Matrix Chain Multiplication reduced computational overhead during data analysis.

Time Complexity:

- Activity Scheduling (Greedy): $O(n)$
- 0/1 Knapsack (DP): $O(n \times W)$

GITHUB LINK <https://github.com/Himavarshini-07/AgroChain-Smart-Agriculture-Supply-Chain-Management-System>

Student Signature	Faculty Signature